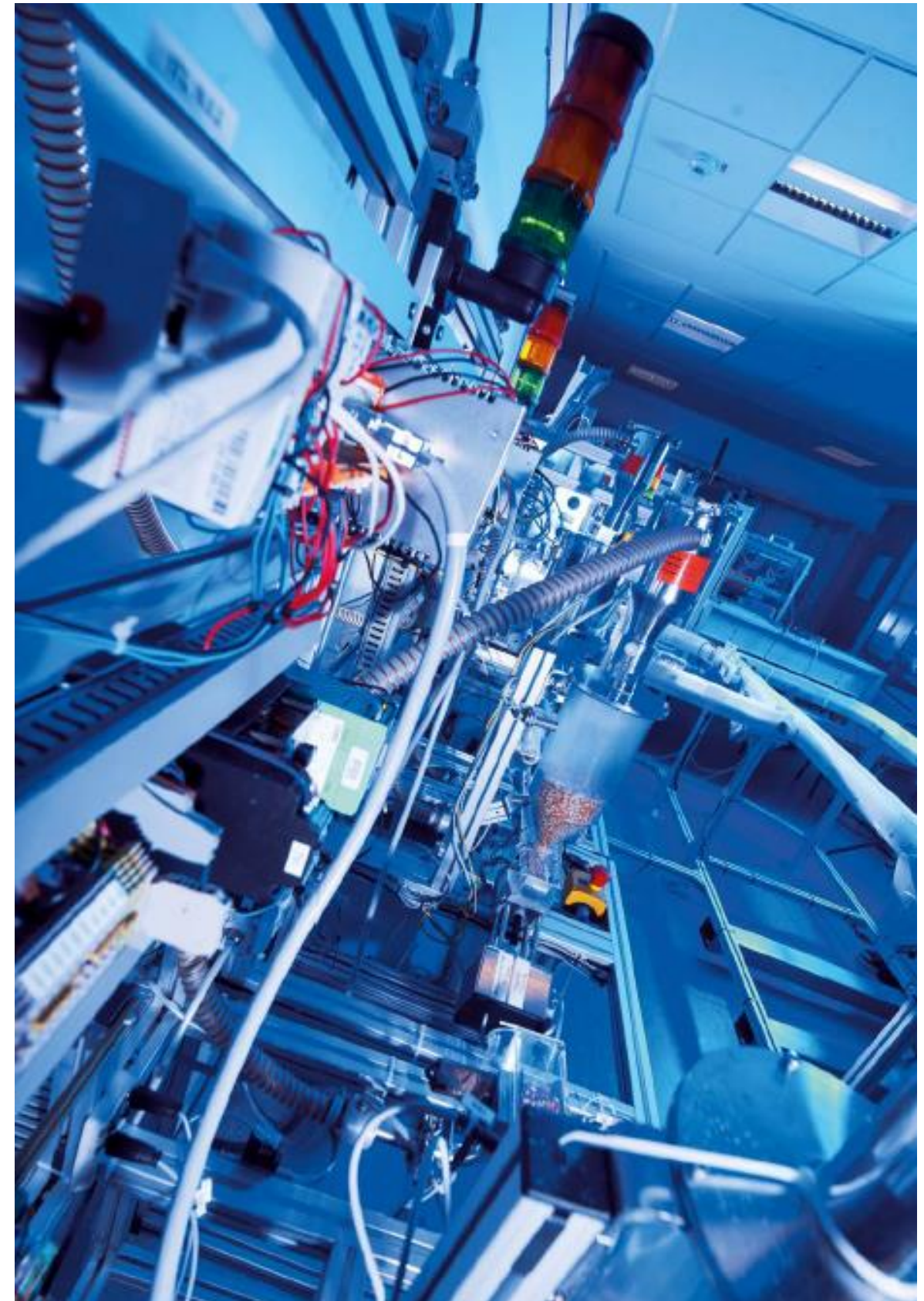


Echtzeit-Datenverarbeitung

Einplanung

Prof. Dr. Henning Trsek

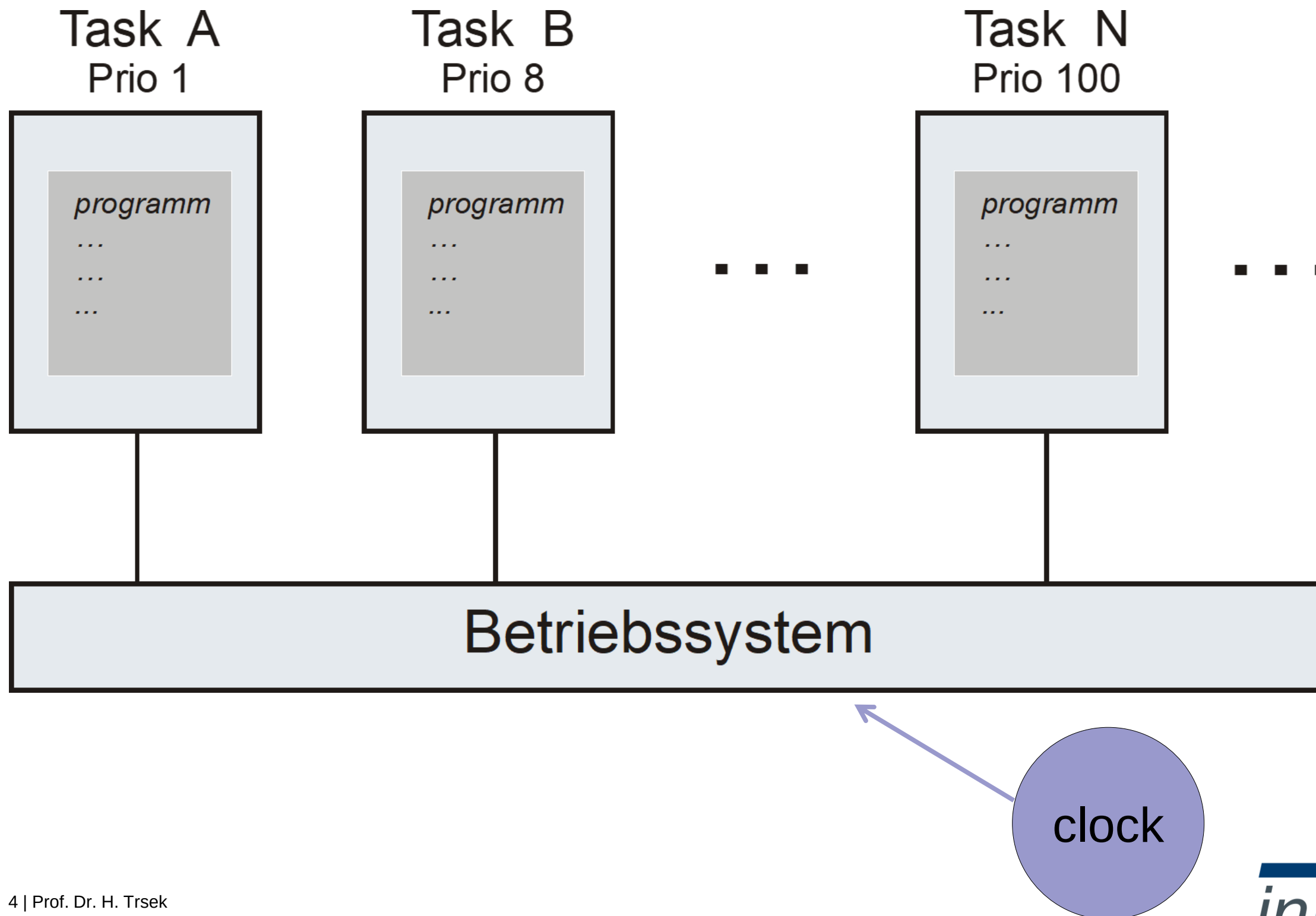
Technische Hochschule Ostwestfalen-Lippe
Fachbereich Elektrotechnik und Technische Informatik
inIT - Institut für industrielle Informationstechnik
henning.trsek@th-owl.de



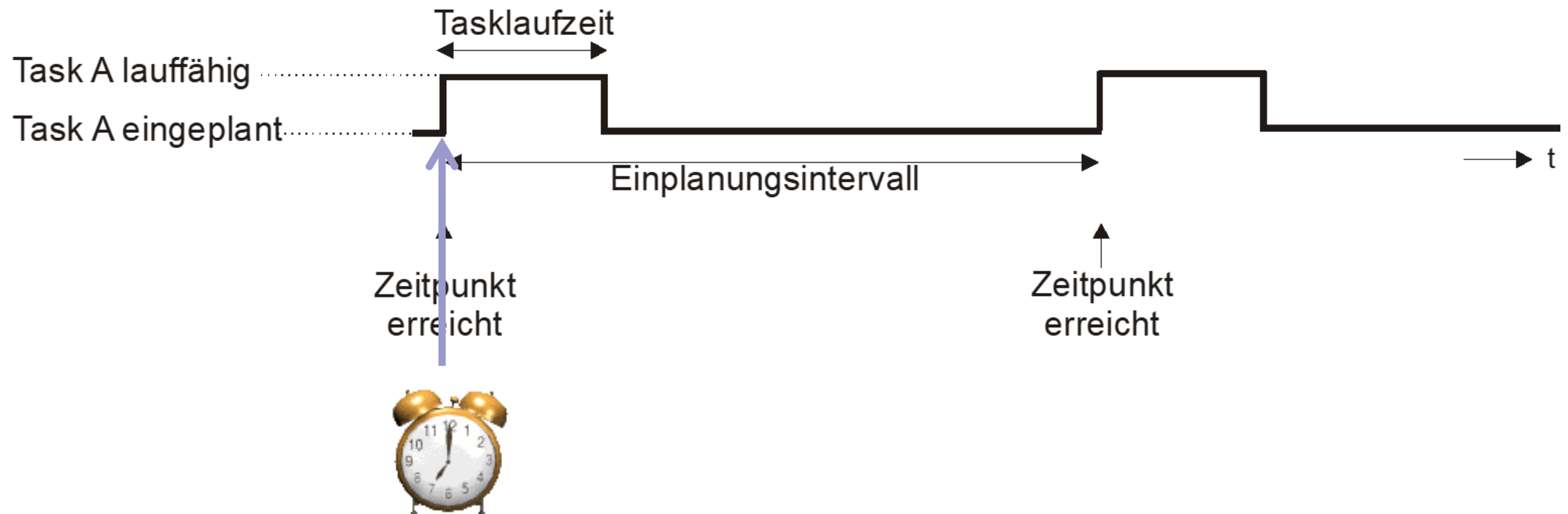
Überblick Peripherie

- ▶ Zyklische Einplanung
- ▶ Einmalige Einplanung
- ▶ Ereignis-basierte Einplanung
- ▶ Einplanung im Multitasking

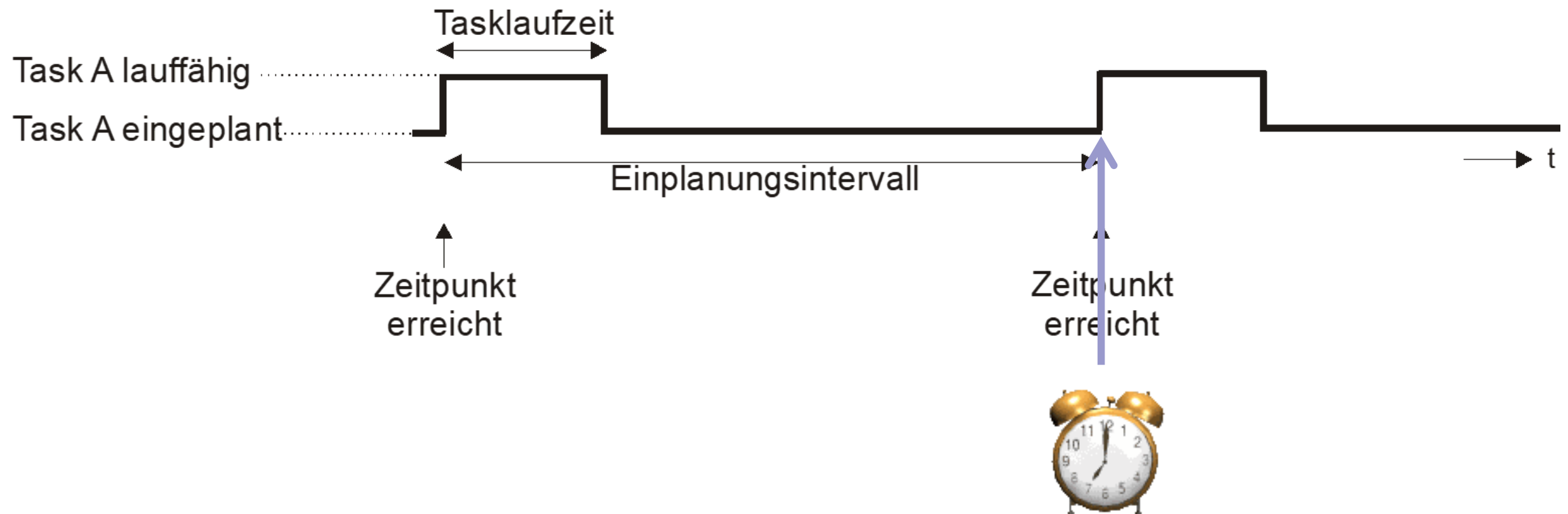
Zeiteinplanung von Tasks



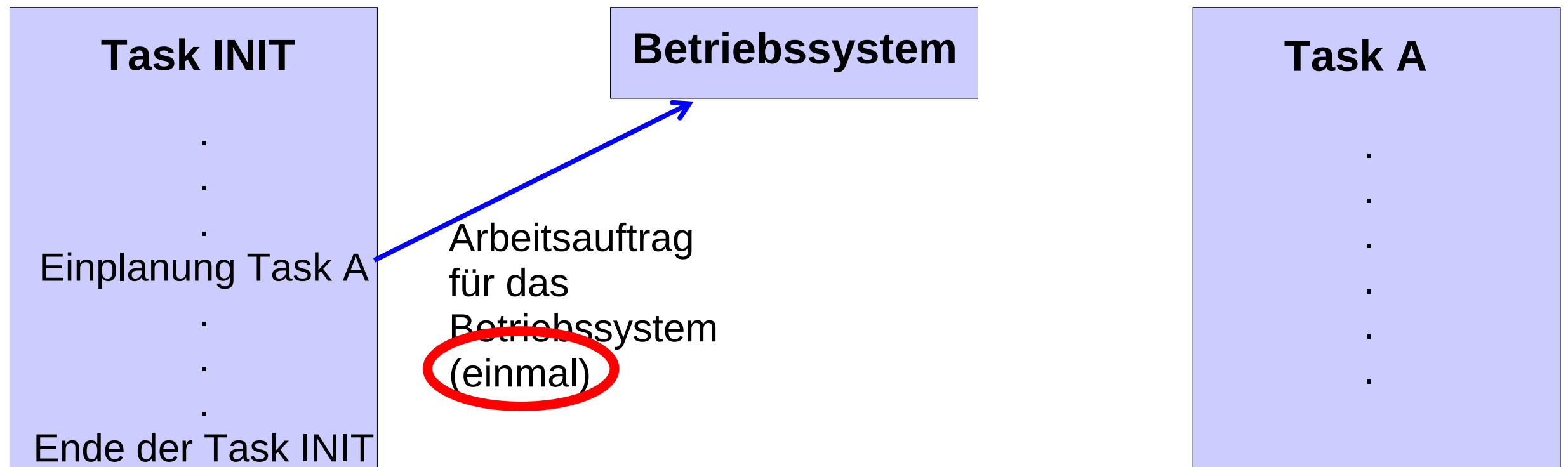
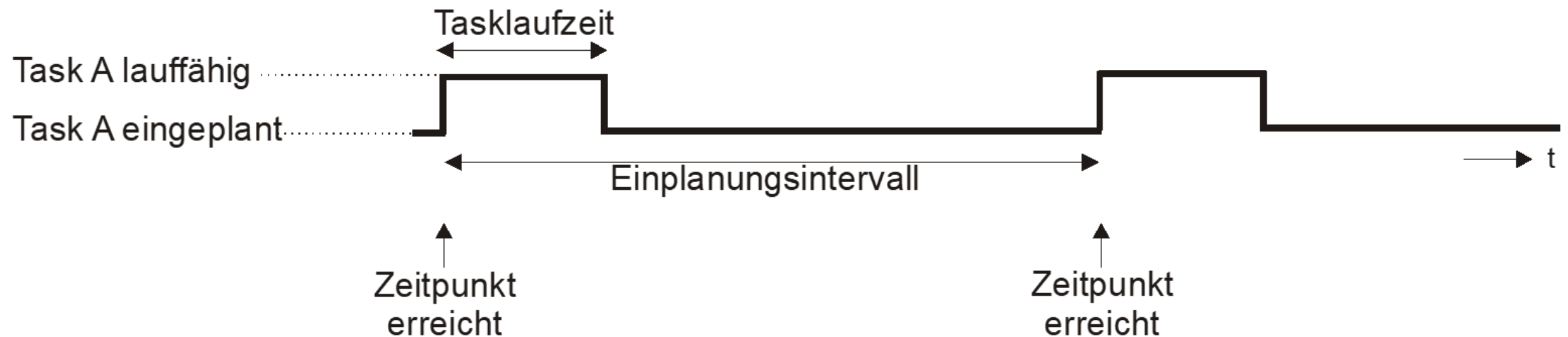
Zeiteinplanung von Tasks: Task wird zyklisch aktiviert



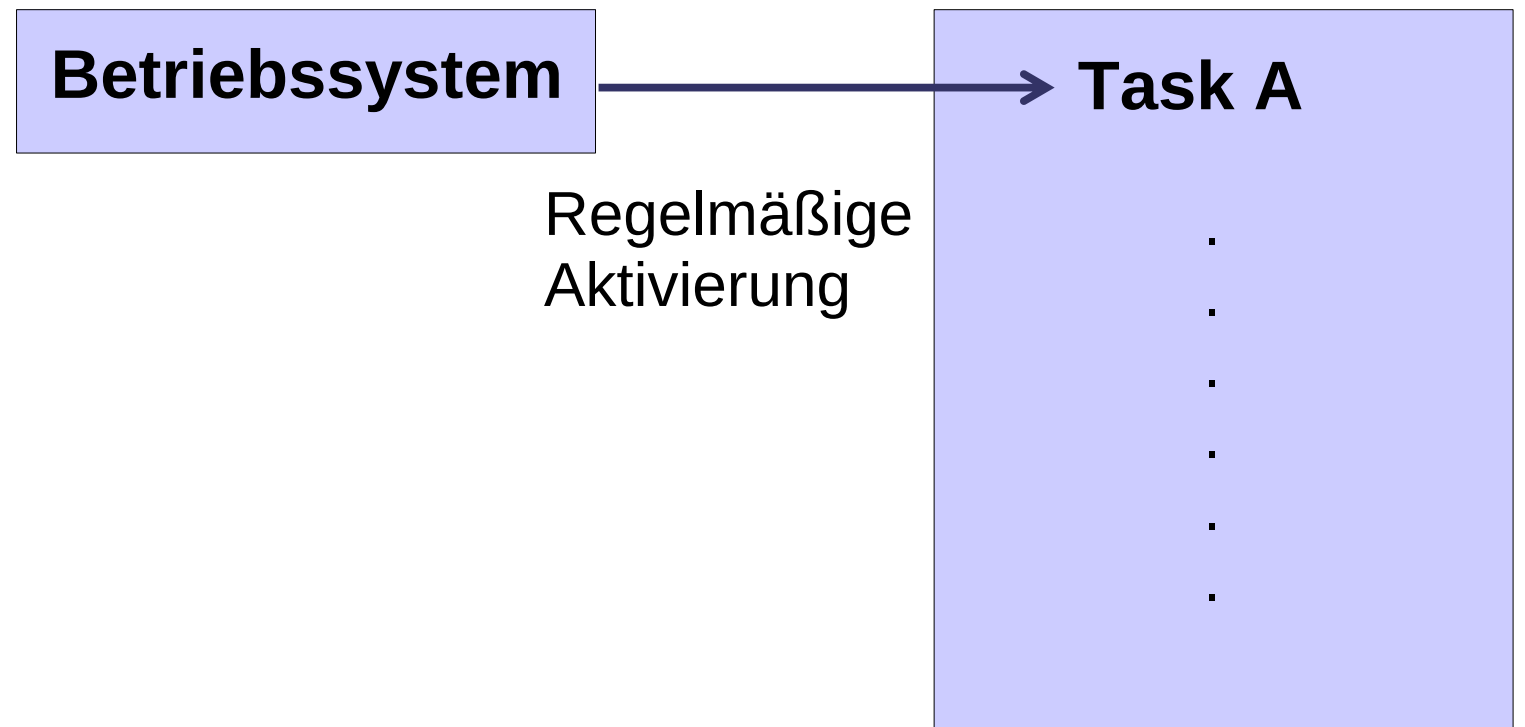
Zeiteinplanung von Tasks: Task wird zyklisch aktiviert



Zeiteinplanung von Tasks: Task wird zyklisch aktiviert

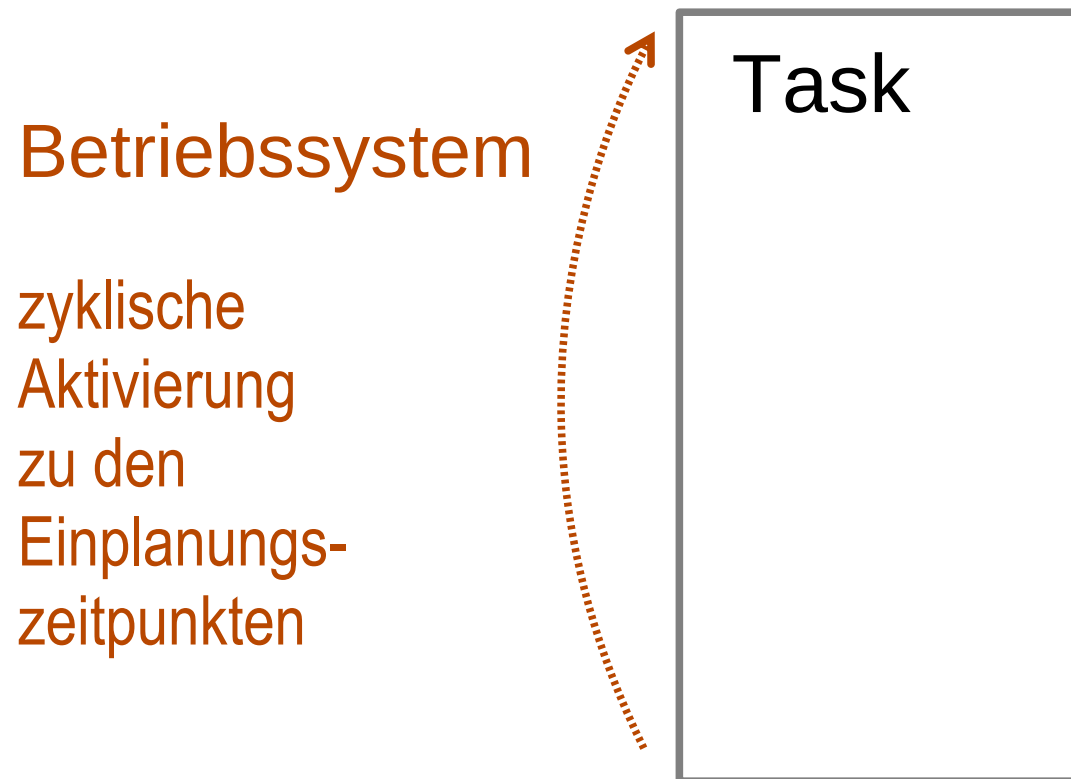


Zeiteinplanung von Tasks: Task wird zyklisch aktiviert



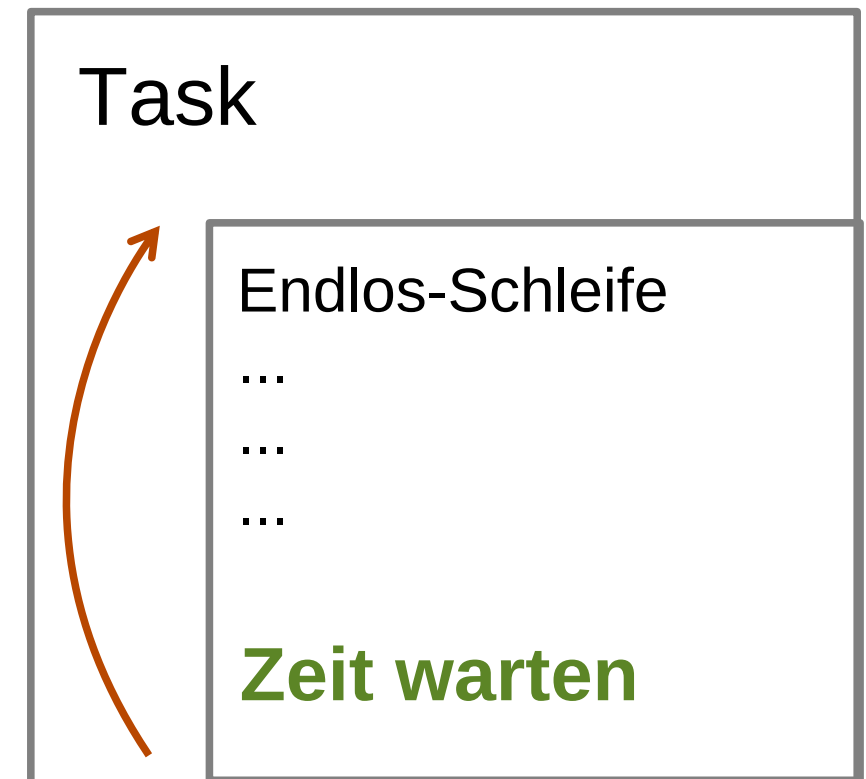
Zeiteinplanung von Tasks: Task wird zyklisch aktiviert

Zyklische Einplanung



In freeRTOS über Softwaretimer und Callbacks

Einmalige Einplanung



In freeRTOS über vTaskDelay

Zyklische Einplanung von Tasks

Zeiteinplanung von Tasks: Task wird zyklisch aktiviert

Main
xTimerCreate()
xTimerStart()

Timer Callback
vTaskResume(taskHandle)

Task

Endlos-Schleife
vTaskSuspend()
...
...
...

```
TimerHandle_t xTimerCreate ( char *  
                             TickType_t  
                             UBaseType_t  
                             void *  
                             TimerCallbackFunction_t pxCallbackFunction );
```

```
BaseType_t xTimerStart ( TimerHandle_t xTimer, TickType_t xBlockTime )
```

ACHTUNG: Alle freeRTOS Timer-Callbacks werden in einer einzelnen Task aufgerufen

→ Wenn ein Callback blockiert oder zu viel Zeit benötigt werden alle Timer verzögert !!!

Zeiteinplanung von Tasks: Task wird zyklisch aktiviert

Main

xTimerCreate()
xTimerStart()

Timer Callback

vTaskResume(taskHandle)

Task

Endlos-Schleife
vTaskSuspend()
...
...
...

```
 BaseType_t xTimerChangePeriod ( TimerHandle_t xTimer,  
                                TickType_t xNewPeriod,  
                                TickType_t xBlockTime );
```

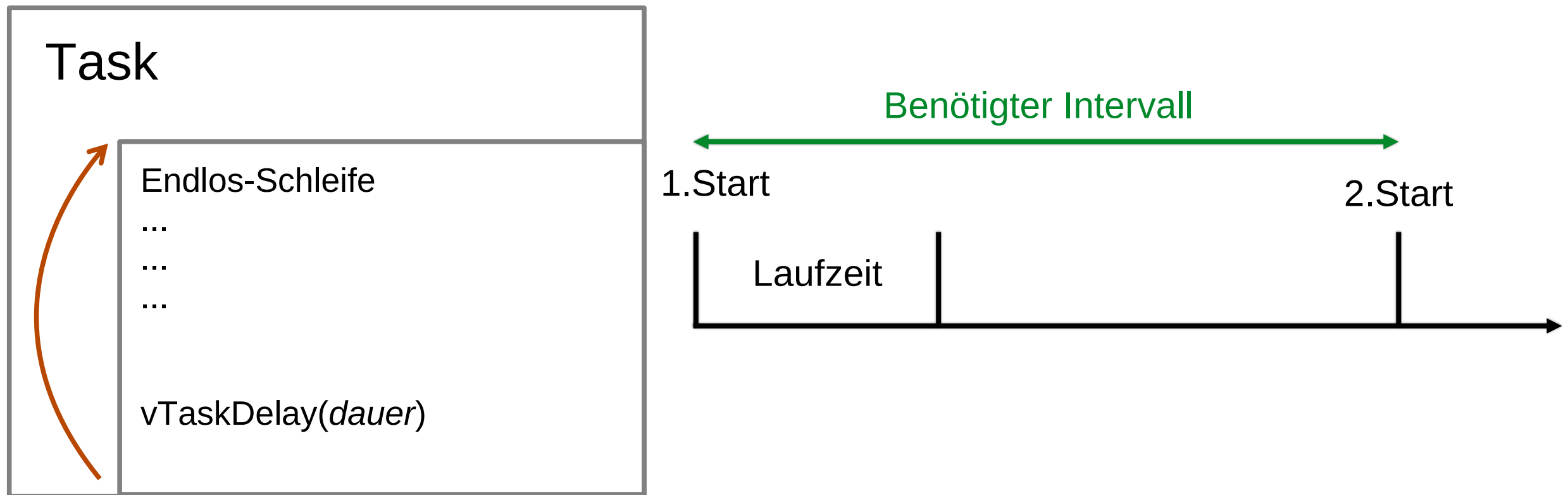
```
 BaseType_t xTimerStop ( TimerHandle_t xTimer, TickType_t xBlockTime )
```

```
 BaseType_t xTimerDelete ( TimerHandle_t xTimer, TickType_t xBlockTime )
```

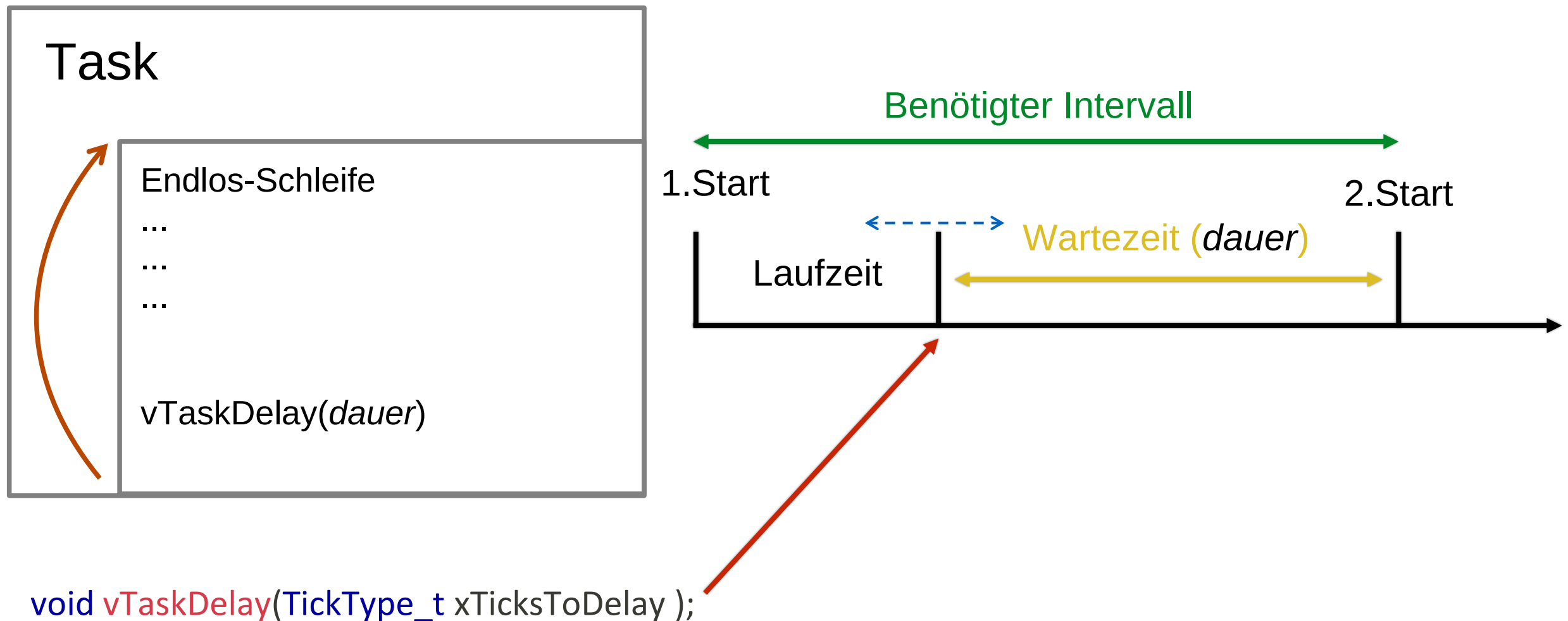
BlockTime beschreibt,
wie viele Ticks auf eine
Antwort des zentralen
Timer Tasks gewartet
werden soll

Einmalige Einplanung von Tasks

Zeiteinplanung von Tasks: Task wird zyklisch aktiviert

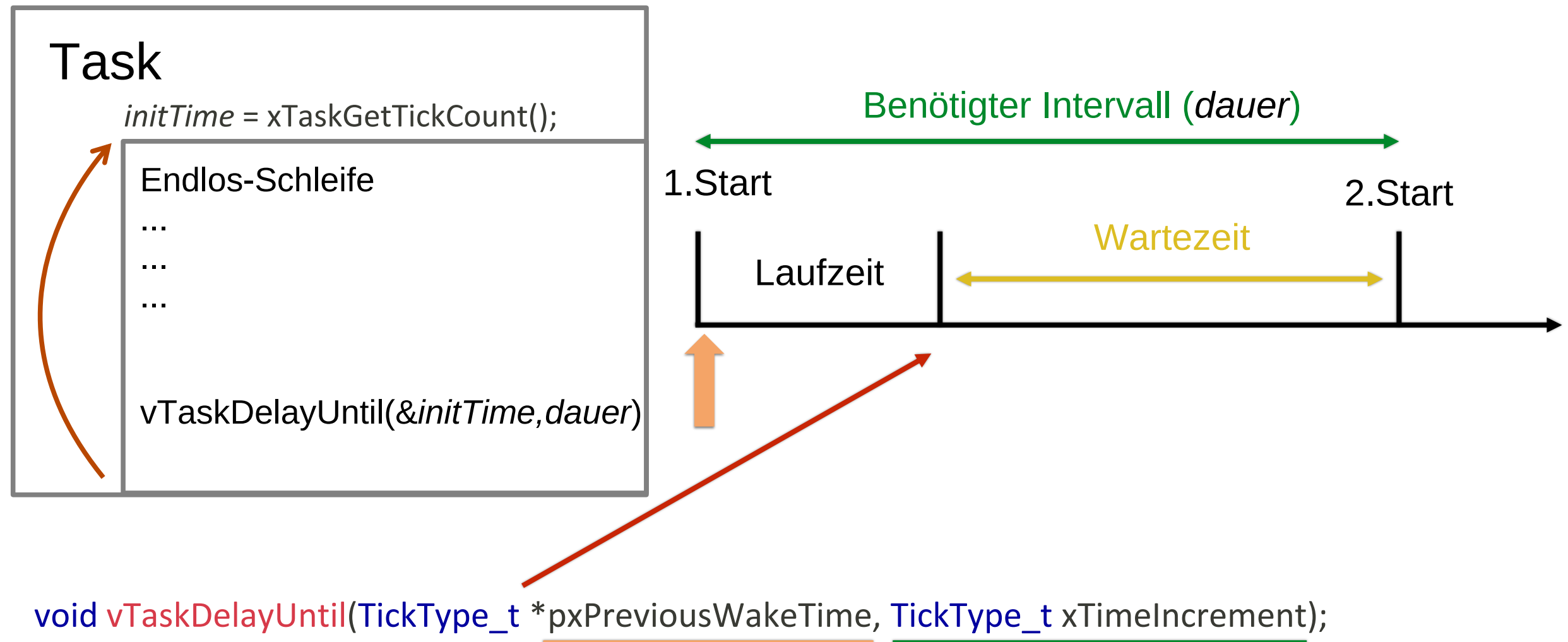


Zeiteinplanung von Tasks: Task wird zyklisch aktiviert



Problem: Tasklaufzeit kann unbekannt oder variabel sein !!!

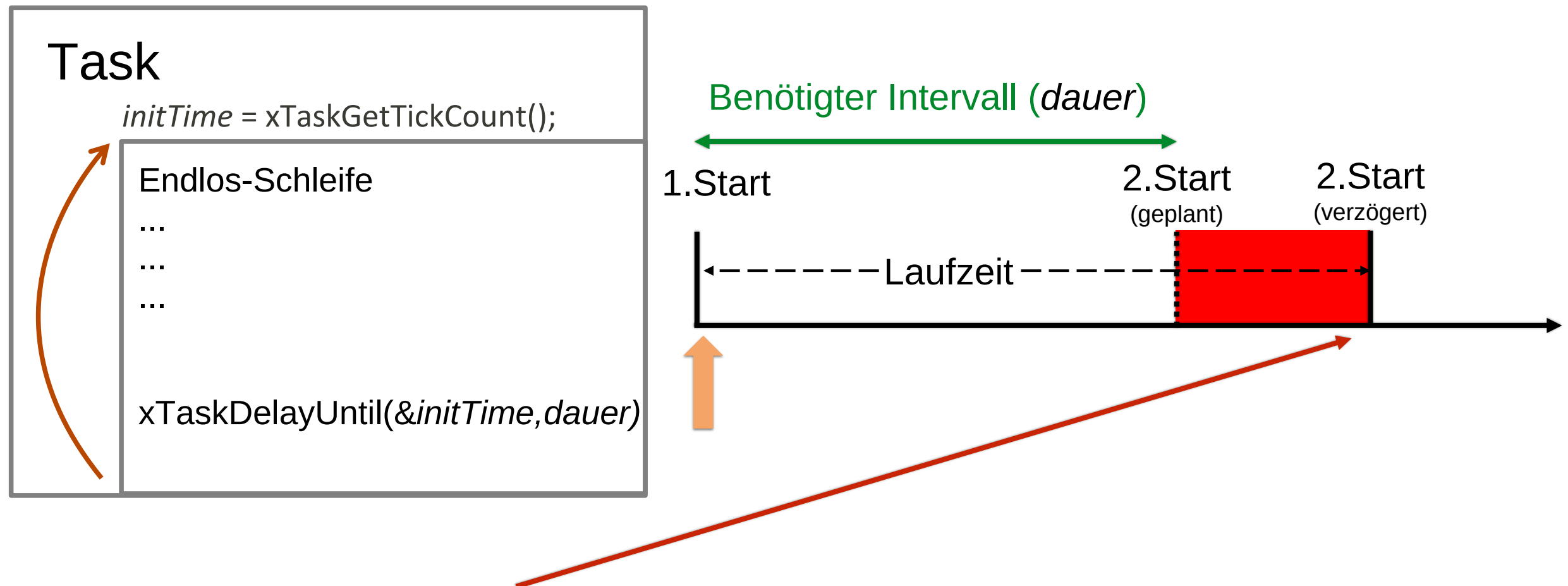
Zeiteinplanung von Tasks: Task wird zyklisch aktiviert



Der Aktivierungszeitpunkt muss nur initial gesetzt werden.
Er wird automatisch von `vTaskDelayUntil` automatisch aktualisiert.

Problem: Tasklaufzeit kann länger als `TickIncrement` sein !!!

Zeiteinplanung von Tasks: Task wird zyklisch aktiviert



`BaseType_t xTaskDelayUntil(TickType_t *pxPreviousWakeTime, TickType_t xTimeIncrement);`

xTaskDelayUntil gibt mit einem Return Wert zurück, ob der Start der Task verzögert wurde. Somit kann die Task auf Verzögerungen reagieren.

(Nicht von der IDE konfigurierbar. Kann wenn benötigt in der FreeRTOSConfig.h aktiviert werden)

Fragen Zeiteinplanung

- ▶ Für welche Zeiteinplanungen reicht es, dass die Uhr grundsätzlich läuft und für welche muss die richtige Uhrzeit eingestellt sein?
- ▶ In der Task *init* soll die Task *regler* zeitzyklisch eingeplant werden, *regler* soll alle 5 Millisekunden aktiviert werden ab sofort. Schreiben Sie die Einplanung.
- ▶ In der Task *regler* mit dem Einplanungsintervall von 10 Millisekunden programmieren Sie eine Wartephase von 30 Millisekunden. Welches Verhalten des Rechners bzw. des Echtzeitbetriebssystems erwarten Sie?
- ▶ Wenn die Task betrieb beendet ist, sollen Teile der Anlage, z.B. ein Lüfter, noch weiterlaufen. Erst nach 10 Sekunden soll die Task ende gestartet werden, die alles abschaltet. Schreiben Sie die erforderliche Zeiteinplanung.